

YANAX CLIENT PORTAL

Diseño técnico de segunda fase

Versión: 1.0 · ERD + permisos + RLS + backlog · 27 de septiembre de 2026

Este documento transforma la especificación funcional en un plano técnico preparado para comenzar implementación, manteniendo el MVP pequeño y seguro.

1. Decisiones de arquitectura

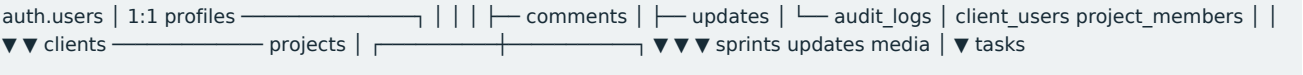
Frontend: React + Vite + TypeScript. UI: Tailwind CSS + shadcn/ui + Lucide. Backend gestionado: Supabase. Persistencia: PostgreSQL. Auth: Supabase Auth. Archivos: Supabase Storage. Autorización: Row Level Security. Deploy: Vercel.

Principio: el frontend nunca decide por sí solo qué datos puede ver un usuario. La base de datos debe imponer la autorización mediante RLS.

2. Modelo de dominio

Entidad	Responsabilidad	Relación
profiles	Perfil y rol de usuario	1:1 con auth.users
clients	Empresa/cliente de Yanax	1:N projects
client_users	Usuarios pertenecientes a un cliente	N:M clients/profiles
projects	Unidad principal contratada	N:1 clients
project_members	Usuarios con acceso a proyectos	N:M projects/profiles
sprints	Bloques de trabajo	N:1 projects
tasks	Trabajo concreto	N:1 sprints
updates	Publicaciones de avance	N:1 projects; opcional sprint
media	Metadatos de archivos	N:1 projects; opcional sprint
comments	Comunicación contextual	N:1 projects
audit_logs	Trazabilidad administrativa	N:1 profiles

3. ERD conceptual



4. Esquema SQL lógico

profiles(id uuid PK → auth.users.id, full_name text, role user_role, avatar_url text, created_at timestamptz) clients(id uuid PK, company_name text, contact_name text, phone text, status client_status, created_at timestamptz) client_users(client_id uuid FK, user_id uuid FK, PK(client_id,user_id)) projects(id uuid PK, client_id uuid FK, name text, description text, status project_status, start_date date, estimated_end date, created_at timestamptz, updated_at timestamptz) project_members(project_id uuid FK, user_id uuid FK, PK(project_id,user_id)) sprints(id uuid PK, project_id uuid FK, name text, description text, status sprint_status, position int, start_date date, end_date date, created_at timestamptz) tasks(id uuid PK, sprint_id uuid FK, title text, description text, status task_status, priority task_priority, position int, created_at timestamptz, updated_at timestamptz) updates(id uuid PK, project_id uuid FK, sprint_id uuid NULL FK, author_id uuid FK, title text, content text, created_at timestamptz, updated_at timestamptz) media(id uuid PK, project_id uuid FK, sprint_id uuid NULL FK, uploaded_by uuid FK, storage_path text, file_name text, mime_type text, size_bytes bigint, created_at timestamptz) comments(id uuid PK, project_id uuid FK, author_id uuid FK, content text, created_at timestamptz, updated_at timestamptz) audit_logs(id uuid PK, actor_id uuid FK, action text, entity_type text, entity_id uuid, metadata

5. Enums y reglas de estado

Tipo	Valores
user_role	admin, client
client_status	active, inactive
project_status	planning, in_progress, paused, completed, archived
sprint_status	pending, in_progress, completed
task_status	pending, in_progress, completed
task_priority	low, medium, high

Regla de progreso: para el MVP se recomienda calcularlo como tareas completadas / tareas totales del proyecto. Esto evita porcentajes arbitrarios.

6. Matriz de permisos

Recurso	Admin	Cliente
Perfil propio	CRUD limitado	R/U propio
Cientes	CRUD	Solo consulta indirecta
Usuarios	Gestión	Solo propio
Proyectos	CRUD	R autorizados
Project members	CRUD	Sin gestión
Sprints	CRUD	R autorizados
Tasks	CRUD	R autorizados
Updates	CRUD	R autorizados
Media	CRUD	R autorizados
Comments	CRUD moderado	CRU propios
Audit logs	R	Sin acceso

R = read, C = create, U = update, D = delete. "Autorizados" significa pertenencia al proyecto correspondiente.

7. Estrategia RLS

Las políticas deben basarse en funciones auxiliares reutilizables. Conceptualmente: `is_admin()` comprueba `profiles.role = 'admin'`; `is_project_member(project_id)` comprueba existencia en `project_members`; `is_client_user_for_project(project_id)` valida la relación del usuario con el cliente propietario.

Ejemplo conceptual: `CREATE POLICY "client can read assigned projects" ON projects FOR SELECT TO authenticated USING ( is_admin() OR EXISTS ( SELECT 1 FROM project_members pm WHERE pm.project_id = projects.id AND pm.user_id = auth.uid() ) );`

Las políticas reales deberán diseñarse evitando recursión entre tablas y funciones `SECURITY DEFINER` mal configuradas. Antes de producción se debe probar acceso cruzado entre clientes.

8. Storage y archivos

Buckets recomendados: `project-media-private` y `project-documents-private`. Rutas lógicas: `projects/{project_id}/sprints/{sprint_id}/{uuid}-{filename}`. Para objetos privados se utilizarán políticas de Storage y/o URLs firmadas.

Validar MIME, tamaño, extensión, nombre seguro y relación entre `storage_path` y proyecto autorizado. Nunca confiar únicamente en la extensión del archivo.

9. Historias de usuario

ID	Historia	Épica	Criterio principal
US-01	Como cliente quiero iniciar sesión para acceder a mis proyectos.	Auth	Login correcto; sesión; errores claros.
US-02	Como cliente quiero ver mis proyectos para conocer su estado.	Dashboard	Solo aparecen proyectos autorizados.
US-03	Como cliente quiero consultar un sprint para conocer qué se desarrolla.	Sprints	Estado, tareas, actualización y evidencias.
US-04	Como cliente quiero ver imágenes/videos/documentos del avance.	Media	Solo archivos autorizados.
US-05	Como cliente quiero comentar un proyecto para comunicar observaciones.	Comentarios	Autor y fecha; acceso autorizado.
US-06	Como admin quiero crear clientes para registrar cuentas.	Admin	Validación y persistencia.
US-07	Como admin quiero crear proyectos y asignar clientes.	Admin	Proyecto aislado al cliente.
US-08	Como admin quiero crear sprints y tareas.	Sprints	Estados y orden funcionales.
US-09	Como admin quiero publicar avances.	Updates	Visible a usuarios autorizados.
US-10	Como admin quiero subir evidencias.	Media	Upload y acceso seguro.
US-11	Como admin quiero responder comentarios.	Comentarios	Respuesta identificable.
US-12	Como cliente quiero contactar a Yanax por WhatsApp.	Contacto	Botón abre número configurado.
US-13	Como admin quiero auditar operaciones críticas.	Auditoría	Actor, acción, entidad y fecha.

10. Backlog priorizado

Prioridad	Items
P0 · Bloqueante	Auth; profiles/roles; clients; projects; project_members; RLS; dashboard cliente.
P1 · MVP esencial	Sprints; tasks; updates; media; comments; admin CRUD; WhatsApp.
P2 · Calidad	Audit logs; paginación; validaciones; monitoring; tests E2E.
P3 · Futuro	Realtime; email; aprobaciones; Gantt; IA; GitHub; SaaS.

11. Orden de implementación

- Fase 1 — Fundación: repositorio, Supabase, variables de entorno, estructura React, tipos, layouts y Auth.
- Fase 2 — Seguridad primero: profiles, roles, clients, project_members y RLS. Probar aislamiento antes de construir muchas pantallas.
- Fase 3 — Dominio: projects → sprints → tasks → updates.
- Fase 4 — Evidencias: Storage + media + vista de documentación.
- Fase 5 — Comunicación: comments + WhatsApp.
- Fase 6 — QA/producción: tests, responsive, errores, auditoría, deploy y dominio.

12. Pruebas de aceptación críticas

- Cliente A no puede leer proyectos del cliente B.
- Cliente A no puede insertar un comentario en proyecto B.
- Cliente A no puede descargar un archivo de proyecto B.
- Cliente no puede crear/editar sprints ni tareas.

- Admin puede gestionar cualquier proyecto.
- Usuario no autenticado no puede consultar datos privados.
- Archivo con MIME/tamaño no permitido es rechazado.
- Archivar un proyecto no elimina accidentalmente su historial.
- El progreso mostrado coincide con la regla definida.
- Las rutas directas del frontend no permiten saltarse RLS.

13. Definition of Ready

- La historia tiene actor, objetivo y criterios de aceptación.
- El modelo de datos necesario existe.
- Las reglas de autorización están definidas.
- El diseño UI mínimo está acordado.
- No depende de una decisión técnica crítica pendiente.

14. Siguiente artefacto

El siguiente paso técnico directo es producir el SQL real de Supabase: extensiones/tipos, tablas, índices, foreign keys, triggers de perfiles, funciones auxiliares de autorización, policies RLS por tabla, policies de Storage y seed de desarrollo. Después podremos generar el esqueleto de React y comenzar Sprint 0.